

Distributed System notes 2

Mục lục phiên dịch

English	Vietnamese
Process	Tiến trình
Thread	Luồng
CPU	Bộ vi xử lý (Central Processing Unit)
Multiprocess	Đa tiến trình
Multithread	Đa luồng
Hyper-Threading	Công nghệ siêu phân luồng
Virtual Processor	Bộ xử lý ảo
Virtual Memory	Bộ nhớ ảo
Context Switching	Chuyển đổi ngữ cảnh
User Mode	Chế độ người dùng
Kernel Mode	Chế độ hạt nhân (Kernel Mode)
User Space Thread	Luồng không gian người dùng
Kernel Thread	Luồng hạt nhân
Light Weight Process (LWP)	Tiến trình nhẹ (LWP)
I/O (Input/Output)	Nhập/xuất (I/O)
Dispatcher	Bộ điều phối
Thread-per-request	Luồng cho mỗi yêu cầu
Thread-per-connection	Luồng cho mỗi kết nối
Thread-per-object	Luồng cho mỗi đối tượng
Concurrency	Đồng thời (Xử lý đồng thời)
OpenMPI	Thư viện OpenMPI trong C
OpenMP	Thư viện OpenMP trong C
Multiprocessing	Đa tiến trình (multiprocessing)
Threading	Quản lý luồng (threads)
Python Multiprocessing	Đa tiến trình trong Python
Python Threads	Luồng trong Python
Go Language	Ngôn ngữ Go

Process, Thread, CPU, multiprocess, multithread

Tiến trình (process): Là 1 chương trình đang được chạy trên một trong các bộ xử lý ảo của hệ điều hành.

Ví dụ của tiến trình chạy:

chương trình: chrome

tiến trình: khi chúng ta bật chrome lên và sử dụng, chrome đang chạy -> trở thành tiến trình

Processes

Name	10% CPU	32% Memory	1% Disk	1% Network	Status	
 Settings	0%	0 MB	0 MB/s	0 Mbps	Suspended	
>  Search (4)	0.1%	32.6 MB	0 MB/s	0 Mbps		
>  Windows Shell Experience Hos...	0%	3.1 MB	0 MB/s	0 Mbps		
>  Google Chrome (24)	0.3%	1,499.3 MB	0.1 MB/s	0.1 Mbps		
>  Obsidian (4)	0.1%	208.9 MB	0.1 MB/s	0 Mbps		
 SnippingTool.exe	0%	2.7 MB	0 MB/s	0 Mbps		
 Microsoft Windows Search Filt...	0%	1.3 MB	0 MB/s	0 Mbps		
 Microsoft Windows Search Pr...	0%	2.0 MB	0 MB/s	0 Mbps		
>  wsappx	0%	3.1 MB	0 MB/s	0 Mbps		
>  wsappx	0%	3.0 MB	0 MB/s	0 Mbps		
>  Service Host: Group Policy	0%	1.3 MB	0 MB/s	0 Mbps		
 UniKey Program	0%	1.9 MB	0 MB/s	0 Mbps		
>  NVIDIA Share (3)	0.1%	71.9 MB	0.1 MB/s	0 Mbps		
>  WebView2 Manager (6)	0%	116.8 MB	0 MB/s	0 Mbps		
>  PostgreSQL Server (7)	0%	12.5 MB	0 MB/s	0 Mbps		
>  mysqld.exe (3)	0%	379.2 MB	0 MB/s	0 Mbps		
>  Host Process (32 bit) (2)	0%	6.7 MB	0 MB/s	0 Mbps		
>  NVIDIA Container (2)	0.1%	61.3 MB	0 MB/s	0 Mbps		
>  Start (2)	0%	47.3 MB	0 MB/s	0 Mbps		
>  Discord (6)	0.1%	453.1 MB	0 MB/s	0 Mbps		
>  NVIDIA Container (4)	0%	89.7 MB	0 MB/s	0 Mbps		
 Windows Audio Device Graph ...	0%	4.5 MB	0 MB/s	0 Mbps		
>  Service Host: Display Enhance...	0%	1.7 MB	0 MB/s	0 Mbps		
>  DS_Lec 2 - Truyen thong.pptx ...	0%	155.1 MB	0 MB/s	0 Mbps		
 Local Artificial Intelligence (AI...	0%	22.3 MB	0 MB/s	0 Mbps		
>  Phone Link	0%	33.6 MB	0 MB/s	0 Mbps		
 Runtime Broker	0%	2.3 MB	0 MB/s	0 Mbps		
 Pentablenet (32 bit)	0%	35.9 MB	0 MB/s	0 Mbps		
>  Service Host: PrintWorkflow_2...	0%	1.3 MB	0 MB/s	0 Mbps		
 COM Surrogate	0%	3.0 MB	0 MB/s	0 Mbps		
>  Service Host: Windows Licens...	0%	2.9 MB	0 MB/s	0 Mbps		
>  Service Host: Unistack Service ...	0%	2.0 MB	0 MB/s	0 Mbps		
 cef_frame_render.exe (32 bit)	0%	111.7 MB	0 MB/s	0 Mbps		
 cef_frame_render.exe (32 bit)	0%	39.8 MB	0 MB/s	0 Mbps		

User OOBE Broker	0%	1.7 MB	0 MB/s	0 Mbps
syzs_dl_svr.exe (32 bit)	0%	1.8 MB	0 MB/s	0 Mbps
> Service Host: DevicesFlow_272...	0%	1.3 MB	0 MB/s	0 Mbps

Hình trên là bảng ví dụ về các tiến trình đang chạy trên máy (hệ điều hành windows, task manager)

Tốc độ xử lý của tiến trình phụ thuộc vào nhiều yếu tố như hiệu năng của CPU, RAM, Disk(ổ cứng), GPU. Nhưng chủ yếu là CPU

Ví dụ:

Intel

CORE

i5

Bộ xử lý Intel® Core™ i5-12400

Bộ nhớ đệm 18M, lên đến 4,40 GHz

Thêm để so sánh

Thông số kỹ thuật

Đặt hàng và tuần thú

Các sản phẩm tương thích

Bản tải xuống

Tài liệu

Hỗ trợ

Thiết yếu

Thiết yếu

Tải xuống thông số kỹ thuật ↓

Thông tin kỹ thuật CPU

Thông tin bổ sung

Thông số bộ nhớ

GPU Specifications

Các tùy chọn mở rộng

Thông số gói

Các công nghệ tiên tiến

Bảo mật & độ tin cậy

Bộ Sơ Tập Sản Phẩm

Tên mã

Phân đoạn thẳng

Số hiệu Bộ xử lý ⓘ

Thuật in thạch bản ⓘ

Giá để xuất cho khách hàng ⓘ

Thông tin kỹ thuật CPU

Số lõi ⓘ

Số P-core

Số E-core

Tổng số luồng ⓘ

Tần số turbo tối đa ⓘ

Tần số Turbo tối đa của P-core ⓘ

Tần số Cơ sở của P-core ⓘ

Bộ nhớ đệm ⓘ

Tổng Bộ nhớ đệm L2

Công suất Cơ bản của Bộ xử lý ⓘ

Công suất Turbo Tối đa ⓘ

Thông tin bổ sung

Bộ xử lý Intel® Core™ i5 thế hệ thứ 12

Alder Lake trước đây của các sản phẩm

Desktop

i5-12400

Intel 7

\$211.00-\$221.00

6

6

0

12

4.40 GHz

4.40 GHz

2.50 GHz

18 MB Intel® Smart Cache

7.5 MB

65 W

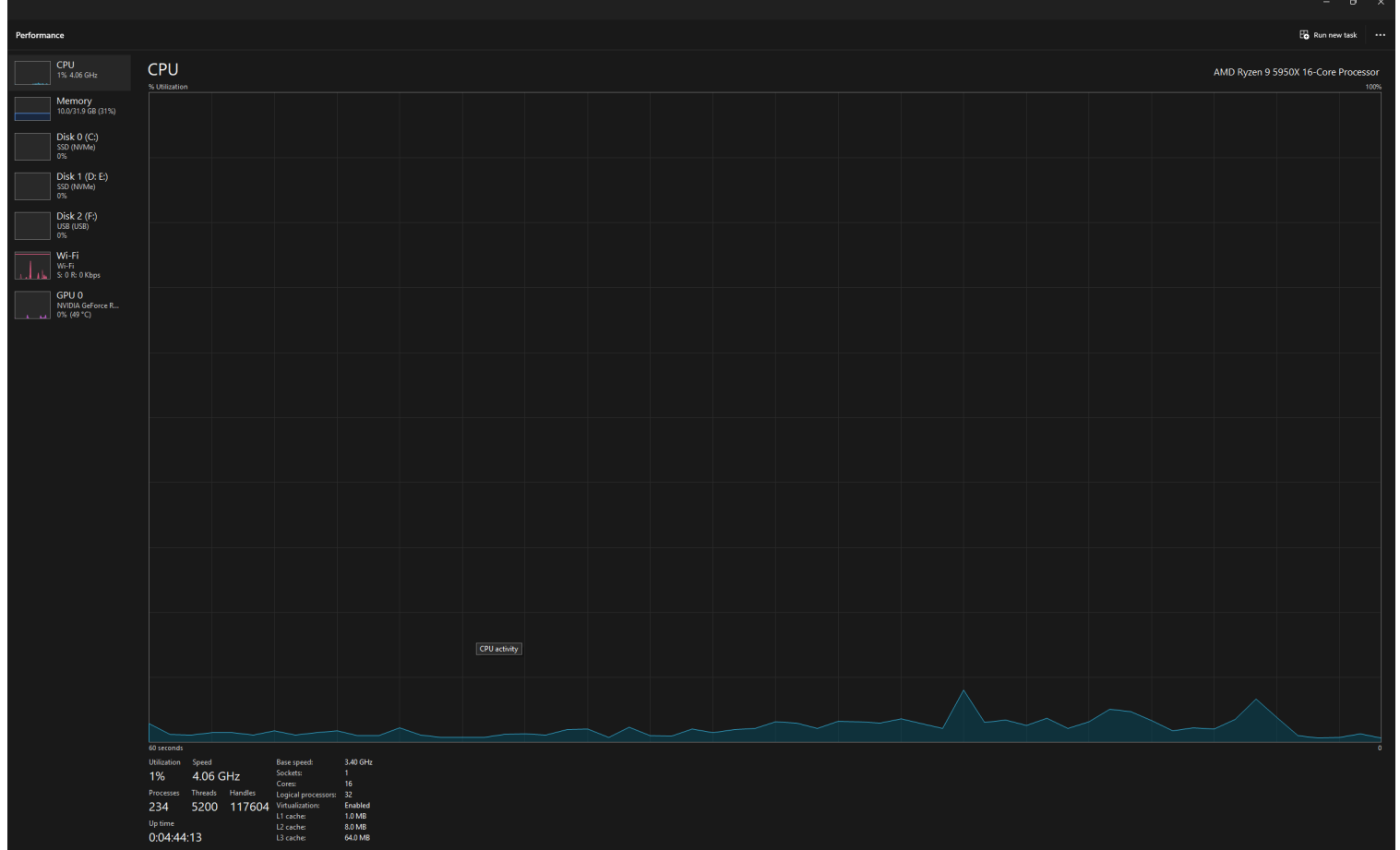
117 W

Số process cores: 6

Số Threads: 12

Có nghĩa là CPU này có 6 lõi vật lý, tức là có thể chạy 6 tiến trình **tại cùng một thời điểm**
Nhờ công nghệ Hyper-Threading, mỗi lõi xử lý 2 luồng, nên tổng cộng CPU có thể xử lý 12 luồng (threads) — tức 12 tiến trình nhẹ có thể chạy song song, dù không hoàn toàn độc lập như 12 lõi thật.

Tuy nhiên, nếu các bạn nhìn vào task manager thì có thể thấy số tiến trình và số threads lớn hơn rất nhiều:



Ví dụ, Cấu hình thấy ở trên CPU Task Manager:

Cores : 16

Logical processes : 32

Processes: 234

Threads: 5200

Về mặt **phần cứng**, có thể chạy 16 tiến trình song song tại cùng một thời điểm (hoặc 32 tiến trình gần như song song). Tuy nhiên, về mặt **phần mềm**, có thể chạy 234 tiến trình (trình duyệt chrome, word, excel, các ứng dụng, chương trình chạy code v.v) và 5200 threads (luồng) đang hoạt động.

=> Máy tính có thể có nhiều process and threads hơn thực tế bởi vì context switching (Chuyển đổi ngữ cảnh) - **Việc CPU tạm dừng một tiến trình đang chạy để chuyển sang chạy tiến trình khác**, hoặc từ một luồng này sang luồng khác. Việc chuyển ngữ cảnh mất khoảng vài μs , tùy thuộc theo kiểu chuyển đổi (luồng/tiến trình). Bởi vì tiến trình không phải lúc nào cũng cần xử lý liên tục nên chúng ta có thể chuyển ngữ cảnh để xử lý tiến trình khác (Review lại môn Operating System)

Hệ điều hành tăng hiệu suất của CPU và RAM bằng cách có Virtual Processor (Bộ xử lý ảo) + Virtual Memory (Bộ nhớ ảo).

Virtual Processor: Một bộ CPU được chia thành các bộ xử lý ảo để có thể chạy các tiến trình một cách dễ dàng, các tiến trình chạy như thể sử dụng một CPU độc lập, tuy rằng đang chia sẻ sử dụng CPU với nhau

Virtual Memory: Sử dụng bộ nhớ trên ổ đĩa cứng làm bộ nhớ tạm thời để mở rộng dung lượng bộ nhớ hệ thống.

Luồng (Thread):

Là một luồng thực thi của tiến trình.

Tiến trình có nhiều luồng thực thi => Tiến trình đa luồng

Các luồng thuộc cùng một tiến trình thì cùng sử dụng không gian bộ nhớ của tiến trình đó

Ví dụ: một App Calculator có thể có luồng giao diện người dùng, một luồng tính toán các phép tính. Hai luồng này đều chia sẻ đến tài nguyên (các biến input người dùng, output)

Một tiến trình có thể có một hoặc nhiều luồng

Khi làm một bài toán lập trình/ hệ phân tán, cần hiểu rõ đa tiến trình (multi-process) và đa luồng (multi) khi nào tốt / xấu để cải thiện hiệu năng

Các hướng tiếp cận cài đặt luồng:

User Mode: Trong chế độ người dùng, các luồng được **quản lý hoàn toàn bởi thư viện hoặc phần mềm người dùng** mà không cần sự can thiệp của hệ điều hành (kernel). Các luồng này không được OS nhận diện là các đơn vị tách biệt, và việc lên lịch thực thi cũng như đồng bộ hóa sẽ được xử lý hoàn toàn trong không gian người dùng.

Tuy nhiên, **khó khăn** ở đây là khi một luồng bị chặn (ví dụ: chờ I/O), cả tiến trình sẽ bị chặn. OS không biết về sự tồn tại của các luồng người dùng, vì vậy nó không thể lên lịch hoặc quản lý sự kiện như I/O. Sự kiện I/O ở đây bao gồm đọc từ đĩa cứng (Disk), đợi tài nguyên từ network (Download file), lấy thông tin từ CSDL, lấy thông tin từ input người dùng (chuột/ bàn phím)

Kernel Mode :

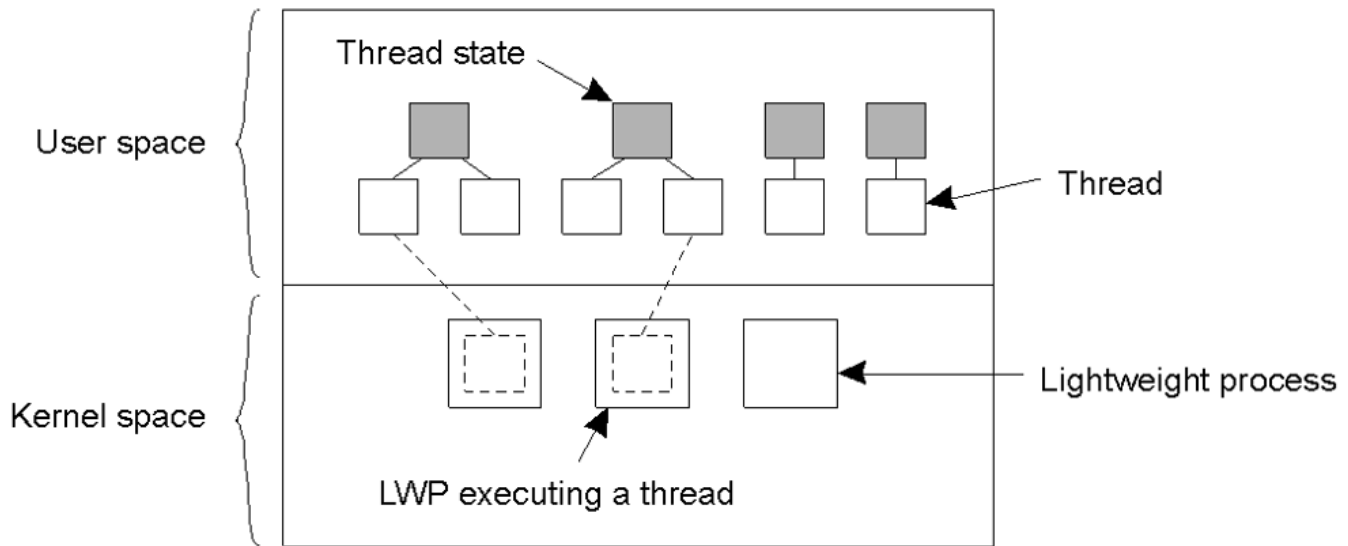
Trong chế độ hạt nhân, các luồng được **quản lý trực tiếp bởi hệ điều hành, chính xác hơn là kernel (nhân) của hệ điều hành**. Hệ điều hành có thể lập lịch, kiểm soát tài nguyên và thực thi các luồng một cách độc lập. Các luồng này có thể thực hiện thao tác hệ thống (như truy xuất bộ nhớ hoặc sử dụng CPU) mà không cần sự can thiệp của người dùng.

Luồng trong chế độ hạt nhân là các **tiến trình con** (kernel threads) do hệ điều hành quản lý và có thể được lên lịch độc lập với các tiến trình khác. Hệ điều hành có thể phân phối CPU cho mỗi luồng, thực hiện điều phối, và xử lý các sự kiện như I/O, giúp các luồng không bị chặn nếu một luồng khác phải chờ.

Light Weight processes :

- Các **LWPs** là một loại luồng mà **hệ điều hành coi là một tiến trình nhỏ**, có thể thực thi độc lập như một tiến trình bình thường nhưng chia sẻ tài nguyên bộ nhớ với các luồng khác trong cùng một tiến trình.
- Một tiến trình (process) có thể có nhiều tiến trình nhẹ (LWP),
- Nếu một LWP trong một process xử lý I/O, LWP đó sẽ bị treo, nhưng không ảnh hưởng đến các LWP khác
- **LWP** kết hợp cả hai yếu tố: được quản lý như một tiến trình (kernel thread) và chia sẻ tài nguyên như một luồng (user thread).

Chung quy: LWP tiết kiệm tốt hơn Kernel mode và đáp ứng được user mode.



Về lí thuyết, trong một tiến trình, có thể sử dụng kết hợp cả LWP và User Thread (User Space) như trên hình để tiết kiệm chi phí trong một tiến trình

Luồng trong hệ phân tán

Giả sử tình huống

Nếu một website chỉ có thể xử lý một yêu cầu tại một thời điểm, các yêu cầu xử lý tuần tự

Ví dụ: người dùng A xem video, không thể xử lý việc khác như viết comment, like bài post. Người dùng B phải đợi người dùng A xem xong mới có thể xem (bởi vì các yêu cầu như xem video hay viết cmt, post bài viết xử lý tuần tự, không phải song song)

==> Cần phải có server đa luồng, trong lúc một luồng

mô hình client server đa luồng

• Client:

- 1 luồng chuyên sinh ra các gói tin request
- 1 luồng khác lo việc đóng gói gói tin và gửi đi cho server

• Server

1 luồng chuyên lo thu nhận các gói tin và xếp chúng vào hàng đợi.
Sau đó gửi chúng đi các luồng khác chuyên xử lý thông điệp. Các luồng này sẽ trực tiếp làm việc với các module vào ra

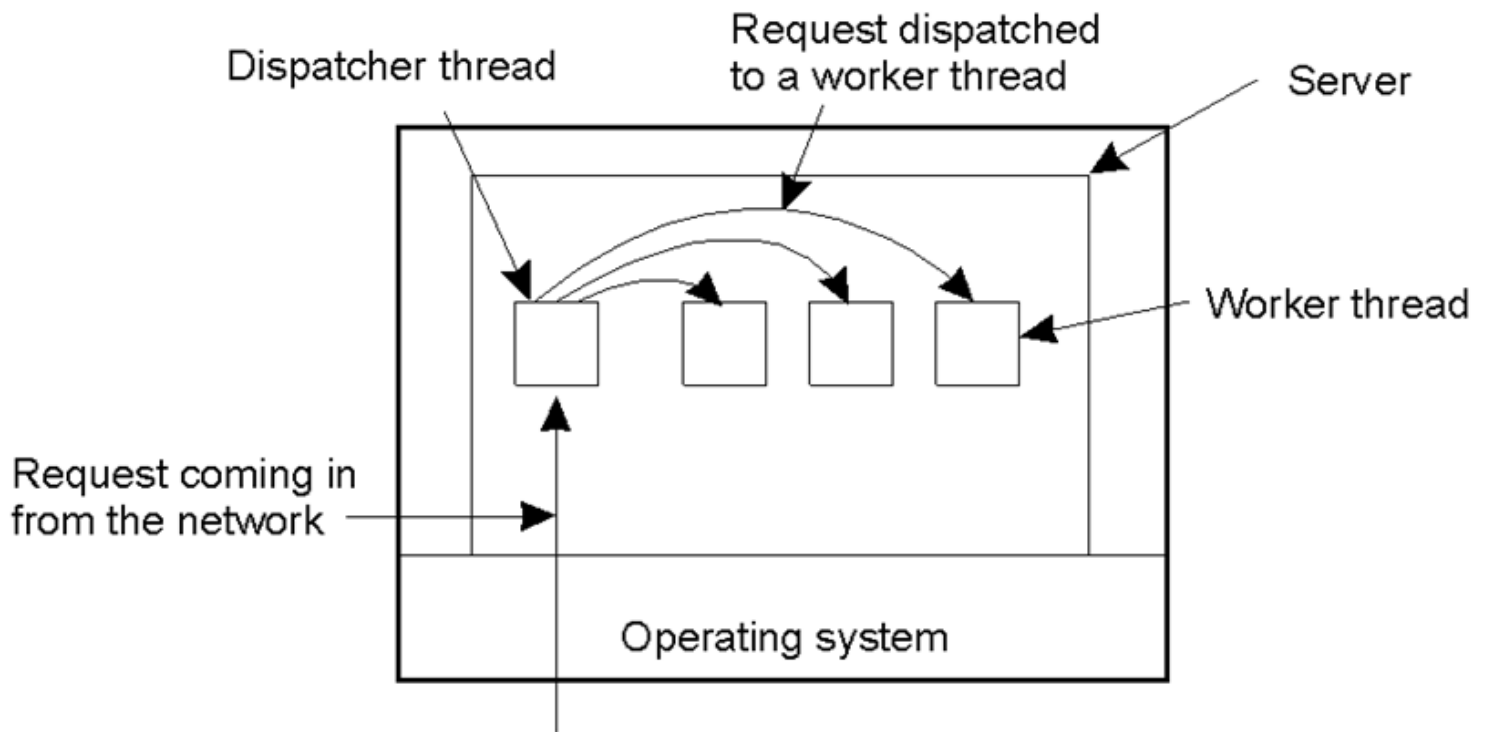
Ý tưởng chính: mỗi luồng làm một tác vụ riêng biệt với nhau.

Mô hình server có điều phối

• Bên cạnh các luồng để xử lý các công việc, tiến trình server có thêm 1 luồng chuyên dụng đảm nhiệm chức năng điều phối các request gọi là Dispatcher.

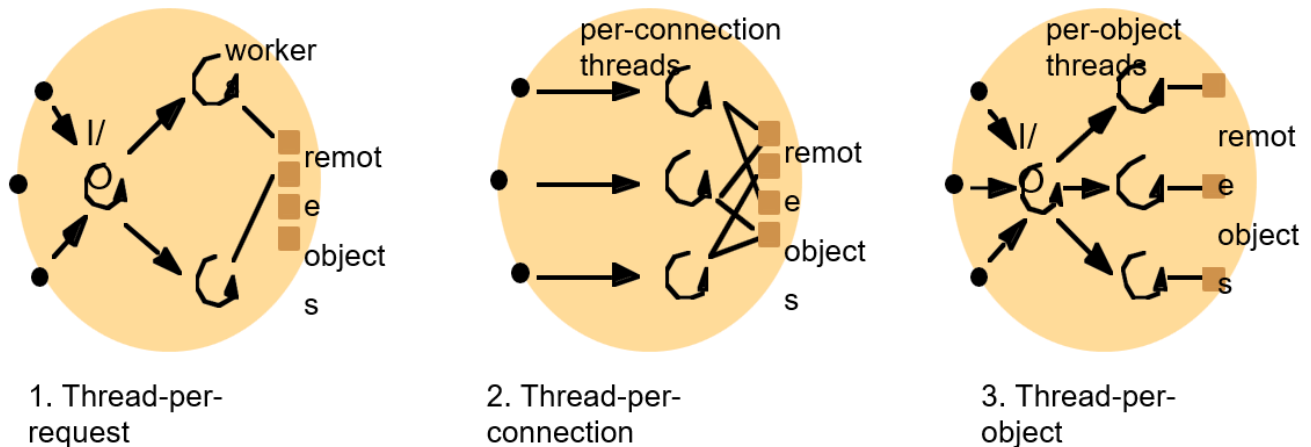
- Mỗi khi nhận được request luồng này sẽ phân tích request đó và gửi đến cho luồng phù hợp để xử lý

=> các luồng hoạt động hiệu quả hơn do mỗi luồng chuyên xử lý các công việc chuyên biệt do luồng điều phối phân loại (Dispatcher) và gửi đến



Ý tưởng chính: Mỗi khi một request đến, chúng ta sẽ có một luồng riêng biệt để điều phối (dispatcher thread), luồng này sẽ điều hướng request đến một luồng khác để xử lý yêu cầu

Ba kiến trúc tổ chức cơ bản cho server đa luồng



1. Thread-per-request (Luồng cho mỗi request):

- Tiến trình vào ra sinh ra 1 luồng worker mới cho mỗi request nhận được
- Mỗi luồng sẽ tự hủy sau khi hoàn thành nhiệm vụ
- Ưu điểm:
 - Không cần có hàng đợi vì mỗi y/c gửi đến đều được sinh ra 1 luồng và xử lý ngay
 - Băng thông có thể đạt mức tối đa vì có khả năng sinh không giới hạn các luồng của worker

- Nhược điểm: việc tạo, hủy luồng trong mỗi request nhận được, đặc biệt trong trường hợp có quá nhiều => giảm hiệu năng và làm tăng chi phí các hoạt động của hệ thống

Ví dụ: mỗi một yêu cầu từ người dùng khi truy cập website chẳng hạn như post bài, like bài viết, xem video, tương tác, download file từ người dùng thì tạo một thread xử lý luôn yêu cầu đó, tự hủy sau khi hoàn thành. Tối đa bằng thông vì xử lý liên tục tạo luồng, nhưng có thể tăng chi phí hệ thống (có thể xử lý từ từ hơn)

2.Thread-per-connection (Luồng cho mỗi kết nối)

Gắn kết mỗi luồng với 1 kết nối.

Khi 1 client gửi request kết nối đến với server, server sẽ tạo ra 1 luồng cho kết nối đó và hủy luồng nếu kết nối đó ngắt

Ví dụ: mỗi một user vào trang chat thì là một thread kết nối

3.Thread-per-object (Luồng cho mỗi đối tượng)

- Tiến trình gắn 1 luồng với 1 đối tượng từ xa (remote object) mà nó quản lý.
- Với kiến trúc này luồng I/O phải quản lý thêm hàng đợi cho mỗi object

Tuy nhiên, giống như Thread-per-connection, các client có thể bị phục vụ với độ trễ cao do 1 số luồng thì phục vụ quá nhiều request trong khi 1 số luồng khác có thể phục vụ ít/không tại cùng thời điểm

Ví dụ:

Một website có thể có các Service như Payment Service, User Service, Authentication Service. Mỗi service là một đối tượng, mỗi user request sẽ được điều phối đến một đối tượng (object) để xử lý.

Áp dụng thực tế

Hiện nay, với sự phát triển của HĐH, process, threads, CPU. Hầu hết threads các bạn được nghe đến trong python, C thì thường là LWP. User space thread không cần thiết nữa

Khi đi làm, đặc biệt trong những công ty lớn, phải tiếp cận và xử lý nhiều nguồn dữ liệu + request từ người dùng + Tính toán lớn => Cần hiểu rõ khi nào thì sử dụng đa tiến trình (multiprocess) và khi nào thì sử dụng đa luồng (multithread)

Có những ngôn ngữ như Go, hỗ trợ concurrency, threads tốt hơn các ngôn ngữ khác. Có những thư viện giúp multiprocess multithread dễ dàng trên các hệ thống lớn như OpenMPI của C

=> Hiểu rõ về thread, process trong từng ngôn ngữ lập trình giúp tiếp cận bài toán dễ dàng hơn

Tham khảo thêm

OpenMP, OpenMPI trong C

multiprocessing, threads trong python

Go

Bài tập

1. Dựa vào bài học, check CPU, GPU, RAM, giải thích về hiệu năng của máy tính mà em đang dùng?

2. Nghiên cứu tìm hiểu, liệt kê ít nhất 12 bài toán phổ biến trong ngành CNTT của em đang học, những bài toán này sử dụng đa luồng, đa tiến trình ở đâu
3. Viết ra giấy rồi chụp ảnh, liệt kê các trường hợp nào thì nên dùng thread, trường hợp nào nên dùng process, khi nào thì nên dùng cả hai. Viết dưới dạng table và đưa ví dụ các bài toán
4. Report, tìm hiểu 1. chatGPT training tập dữ liệu lớn bằng distributed system như thế nào. Đưa link nguồn tài liệu tham khảo từ google

Tạo blog với tên: Tiến trình & Luồng